LAB-3

PostgreSQL 18.6 Controlled Failback
Restore Original Primary & Standby Roles on AWS EC2

Document Type	Technical Lab SOP / Class Notes
Platform	AWS EC2 - Amazon Linux 2023
Database	PostgreSQL 18.6
Replication	Native Physical Streaming Replication - Asynchronous
Tested By	Praveen Madupu
Lab Date	02 September 2026 - Wednesday
Final Status	PASS - Original roles restored successfully

Final topology: DB1 PRIMARY (172.30.2.177) -> DB2 HOT STANDBY (172.30.2.128)

Index

1. Purpose and Lab Outcome

LAB-3 performs a controlled failback after LAB-2. LAB-2 left DB2 as Primary and DB1 as Hot Standby. This lab safely synchronizes the former primary, stops DB2, promotes DB1, rebuilds DB2 from the new DB1 primary, and restores the original replication direction.

Note: This is a controlled role restoration using native PostgreSQL physical streaming replication. The tested lab used a fresh pg_basebackup to rebuild the former primary. It did not use pg_rewind.

2. Lab Environment

Node	Hostname	Private IP	Version	LAB-3 Role
DB1	db1	172.30.2.177	PostgreSQL 18.6	Target / Final Primary
DB2	db2	172.30.2.128	PostgreSQL 18.6	Starting Primary / Final Standby

PGDATA: /var/lib/pgsql/data
Service: postgresql.service
Replication role: replicator
Lab database: labdb
Validation table: replication_validation
Replication mode: asynchronous physical streaming replication

3. Starting and Target Architecture

```
START OF LAB-3

DB2 PRIMARY
172.30.2.128
Read / Write
        |
        | Asynchronous Physical WAL Streaming
        v
DB1 HOT STANDBY
172.30.2.177
Read Only

TARGET / FINAL STATE

DB1 PRIMARY
172.30.2.177
Read / Write
Timeline 3
        |
        | Asynchronous Physical WAL Streaming
        v
DB2 HOT STANDBY
172.30.2.128
Read Only
```

4. Prerequisites and Safety Rules

- LAB-2 must already be complete: DB2 is Primary and DB1 is a healthy streaming standby.
- Both nodes must use the same PostgreSQL major version and belong to the same database system.
- Confirm DB1 has received and replayed the final DB2 WAL before stopping DB2.

- DB2 must remain stopped while its former-primary data directory is preserved and rebuilt.
- Never allow both diverged nodes to accept writes at the same time. Fencing prevents split-brain.
- The tested lab uses a fresh base backup and therefore requires enough disk space for the preserved old PGDATA plus the new copy.

5. Step 1 - Validate DB2 as Current Primary

Before failback, confirm that DB2 is the writable primary and is streaming to DB1.

```
hostname
hostname -I
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_setting('transaction_read_only') AS read_only,
    pg_current_wal_lsn() AS current_wal_lsn;
"

sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn,replay_lsn)) AS replication_lag
FROM pg_stat_replication;
"
```

Expected / validated result: DB2 returned pg_is_in_recovery() = f, read_only = off, DB1 = 172.30.2.177, state = streaming, mode = async, lag = 0 bytes.

6. Step 2 - Validate DB1 as Current Standby

Confirm DB1 is still in recovery, read-only, and receiving WAL from DB2.

```
hostname
hostname -I
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_setting('transaction_read_only') AS read_only,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn;
"

sudo -u postgres psql -c "
SELECT
    status,
    sender_host,
    sender_port,
    latest_end_lsn,
    slot_name
FROM pg_stat_wal_receiver;
"
```

Expected / validated result: DB1 returned is_standby = t, read_only = on, and WAL receiver streaming from db2:5432.

7. Step 3 - Create Pre-Failback Validation Transaction

Create a known transaction on DB2 before the controlled role change. This provides a clear data marker that must reach DB1.

```
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-3: Before Controlled Failback');
"

sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id DESC
LIMIT 5;
"
```

Expected / validated result: Marker LAB-3: Before Controlled Failback was created successfully.

8. Step 4 - Synchronize Final Transaction and Confirm Zero Lag

Create the final transaction on DB2, capture its WAL LSN, then verify DB1 has received and replayed through that point before shutting DB2 down.

```
# On DB2
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-3: Final Transaction Before DB2 Shutdown');
"

sudo -u postgres psql -Atc "
SELECT pg_current_wal_lsn();
"

sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn,replay_lsn)) AS replication_lag
FROM pg_stat_replication;
"

# On DB1
sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id DESC
LIMIT 5;
"

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn;
"
```

Expected / validated result: DB2 final LSN was 0/6000ED8. DB1 receive_lsn and replay_lsn both reached 0/6000ED8; the final marker was visible on DB1.

Note: Run pg_last_wal_replay_lsn() comparisons on the standby. Running it on the primary does not validate the current

primary's own WAL position.

9. Step 5 - Stop and Fence DB2

Once DB1 is fully synchronized, stop DB2 and confirm PostgreSQL is no longer listening. This prevents the old primary from accepting writes after DB1 is promoted.

```
# On DB2
sudo systemctl stop postgresql
sudo systemctl is-active postgresql

sudo ss -lntp | grep 5432 || echo "DB2 PostgreSQL port 5432 is DOWN"

ps -ef | grep '[p]ostgres'
```

Expected / validated result: DB2 service = inactive, port 5432 down, and no PostgreSQL server processes remained.

10. Step 6 - Promote DB1 to Primary

With DB2 fenced, promote DB1. pg_promote(wait => true) waits for promotion to complete before returning.

```
# On DB1
sudo -u postgres psql -c "
SELECT pg_promote(wait => true, wait_seconds => 60);
"

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_setting('transaction_read_only') AS read_only,
    pg_current_wal_lsn() AS current_wal_lsn;
"
```

Expected / validated result: pg_promote returned t. DB1 changed to pg_is_in_recovery() = f and transaction_read_only = off.

11. Step 7 - Validate DB1 Primary and Timeline

Verify the promotion created a new timeline and prove that DB1 accepts writes.

```
sudo -u postgres psql -c "
SELECT
    timeline_id,
    redo_lsn
FROM pg_control_checkpoint();
"

sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-3: DB1 Restored as Primary');
"

sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id DESC
LIMIT 5;
"
```

Expected / validated result: DB1 was on timeline 3 and accepted the marker LAB-3: DB1 Restored as Primary.

12. Step 8 - Prepare DB1 for DB2 Replication Connection

DB1 is now the sending server. Confirm the replication role and pg_hba.conf rule permit DB2 (172.30.2.128) to connect as replicator.

```
sudo -u postgres psql -c "
SELECT
    rolname,
    rolcanlogin,
    rolreplication
FROM pg_roles
WHERE rolname = 'replicator';
"

sudo grep -n "172.30.2.128" /var/lib/pgsql/data/pg_hba.conf

sudo systemctl reload postgresql
sudo systemctl is-active postgresql
```

Expected / validated result: replicator had LOGIN and REPLICATION privileges. The tested HBA included host replication replicator 172.30.2.128/32 scram-sha-256.

13. Step 9 - Prepare DB2 Authentication and Connectivity

Keep DB2 PostgreSQL stopped. Configure the postgres account's password file and verify DB1 is reachable through the replication protocol.

```
# On DB2
sudo bash -c 'cat > /var/lib/pgsql/.pgpass <<EOF
db1:5432:replication:replicator:ReplLab@2026
172.30.2.177:5432:replication:replicator:ReplLab@2026
EOF'

sudo chown postgres:postgres /var/lib/pgsql/.pgpass
sudo chmod 600 /var/lib/pgsql/.pgpass

getent hosts db1

sudo -u postgres psql "host=db1 port=5432 user=replicator replication=true passfile=/var/lib/pgsql/.pgpass" -c
"IDENTIFY_SYSTEM;"
```

Expected / validated result: DB1 returned system ID 7680977547376578577 and timeline 3.

Note: The lab's earlier IDENTIFY_SYSTEM invocation prompted for a password despite the .pgpass file. The SOP makes the passfile explicit in the connection string to remove ambiguity.

14. Step 10 - Preserve Former DB2 Primary PGDATA

Do not reuse DB2's diverged former-primary cluster as-is. Preserve it for rollback/reference, then create a clean empty PGDATA owned by postgres.

```
sudo systemctl is-active postgresql

sudo mv /var/lib/pgsql/data /var/lib/pgsql/data_old_primary_lab3_20260902

sudo mkdir -p /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data

sudo ls -ld /var/lib/pgsql/data
sudo ls -ld /var/lib/pgsql/data_old_primary_lab3_20260902
sudo ls -la /var/lib/pgsql/data
```

Expected / validated result: New PGDATA was empty and mode 700; the old DB2 primary was preserved at /var/lib/pgsql/data_old_primary_lab3_20260902.

15. Step 11 - Rebuild DB2 from DB1 with pg_basebackup

Create a fresh physical copy from DB1. -Fp uses plain format, -Xs streams required WAL, -P shows progress, and -R prepares the copy as a standby.

```
sudo -u postgres pg_basebackup   -h db1   -p 5432   -U replicator   -D /var/lib/pgsql/data   -Fp   -Xs   -P   -R
```

Expected / validated result: 32410/32410 kB (100%), 1/1 tablespace.

16. Step 12 - Validate standby.signal and primary_conninfo

Before starting DB2, verify the copied cluster is PostgreSQL 18 and that -R created standby.signal. Inspect primary_conninfo carefully.

```
sudo ls -l /var/lib/pgsql/data/standby.signal
sudo cat /var/lib/pgsql/data/PG_VERSION

sudo grep -n "primary_conninfo" /var/lib/pgsql/data/postgresql.auto.conf
```

Expected / validated result: standby.signal existed, PG_VERSION = 18. The file contained an inherited stale host=db2 entry and the new correct host=db1 entry.

17. Step 13 - Clean Stale primary_conninfo

Because the source DB1 copy contained its historical auto-configuration, back up postgresql.auto.conf and remove only the stale host=db2 entry. Keep the correct host=db1 entry.

```
sudo cp /var/lib/pgsql/data/postgresql.auto.conf /var/lib/pgsql/data/postgresql.auto.conf.before_lab3_cleanup

sudo chown postgres:postgres /var/lib/pgsql/data/postgresql.auto.conf.before_lab3_cleanup

sudo bash -c 'ls -l /var/lib/pgsql/data/postgresql.auto.conf*'

sudo grep -n "primary_conninfo" /var/lib/pgsql/data/postgresql.auto.conf.before_lab3_cleanup

sudo sed -i "/primary_conninfo.*host=db2 /d" /var/lib/pgsql/data/postgresql.auto.conf

sudo grep -n "primary_conninfo" /var/lib/pgsql/data/postgresql.auto.conf

sudo cat /var/lib/pgsql/data/postgresql.auto.conf
```

Expected / validated result: Only the host=db1 primary_conninfo remained.

Note: When a wildcard path is under a protected directory, use sudo bash -c 'ls ...*' so the wildcard is expanded by the privileged shell.

18. Step 14 - Start DB2 as Hot Standby

After configuration cleanup, start DB2 and verify recovery/read-only state.

```
sudo ls -l /var/lib/pgsql/data/standby.signal
sudo cat /var/lib/pgsql/data/PG_VERSION
sudo systemctl is-active postgresql

sudo systemctl start postgresql
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_setting('transaction_read_only') AS read_only,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn;
"
```

Expected / validated result: DB2 became active with is_standby = t, read_only = on, receive_lsn = replay_lsn = 0/8000168.

19. Step 15 - Validate WAL Receiver on DB2

pg_stat_wal_receiver must show DB2 streaming from DB1.

```
sudo -u postgres psql -c "
SELECT
    status,
    sender_host,
    sender_port,
    latest_end_lsn,
    slot_name
FROM pg_stat_wal_receiver;
"
```

Expected / validated result: status = streaming, sender_host = db1, sender_port = 5432, latest_end_lsn = 0/8000168.

20. Step 16 - Validate WAL Sender and Zero Lag on DB1

On the new primary, pg_stat_replication must show DB2 connected and caught up.

```
sudo -u postgres psql -c "
SELECT
    application_name,
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(
        pg_wal_lsn_diff(sent_lsn,replay_lsn)
    ) AS replication_lag
FROM pg_stat_replication;
"
```

Expected / validated result: DB1 showed DB2 at 172.30.2.128, streaming, async, all LSNs 0/8000168, replication_lag = 0 bytes.

21. Step 17 - Final End-to-End Data Validation

Insert a final marker on DB1 and verify the exact row appears on DB2. Confirm DB2 remains read-only.

```
# On DB1
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-3: DB1 Primary to DB2 Standby Final Validation');
"
sudo -u postgres psql -d labdb -c "
SELECT
    id,
    message,
    created_at
FROM replication_validation
ORDER BY id DESC
LIMIT 5;
"

# On DB2
sudo -u postgres psql -d labdb -c "
SELECT
    id,
    message,
    created_at
FROM replication_validation
ORDER BY id DESC
LIMIT 5;
"
sudo -u postgres psql -d labdb -c "
SHOW transaction_read_only;
"
```

Expected / validated result: Row ID 73, LAB-3: DB1 Primary to DB2 Standby Final Validation, was visible on DB2. DB2 transaction_read_only = on.

22. Final Acceptance Checklist

Acceptance Criterion	Result
DB1 PostgreSQL active	**PASS**
DB1 is primary / not in recovery	**PASS**
DB1 transaction_read_only = off	**PASS**
DB1 promotion timeline = 3	**PASS**
DB2 PostgreSQL active	**PASS**
DB2 is in recovery	**PASS**
DB2 transaction_read_only = on	**PASS**
DB2 WAL receiver streaming from db1	**PASS**
DB1 sees DB2 at 172.30.2.128	**PASS**
Replication mode = async	**PASS**
Observed final replication lag = 0 bytes	**PASS**
Final validation row visible on DB2	**PASS**
Original DB1 -> DB2 topology restored	**PASS**

23. Troubleshooting Notes from the Tested Lab

Observation	Correct Practice
Do not run pg_last_wal_replay_lsn() on the primary to judge current primary WAL progress.	Use pg_current_wal_lsn() on a primary; use receive/replay functions on a standby.
pg_basebackup target directory must be empty.	Stop PostgreSQL, preserve/move the old PGDATA, create an empty postgres-owned directory, then run the base backup.
A copied postgresql.auto.conf can contain historical primary_conninfo.	Back it up and ensure the final standby has one correct upstream connection entry.
Shell wildcard with sudo can fail before sudo is applied.	Use sudo bash -c 'ls -l /protected/path/file*'.
Identity/sequence values may have gaps.	Do not use gapless IDs as a replication-health test; validate committed rows and WAL positions instead.

24. Production Considerations

- Use a formal fencing/STONITH mechanism or HA manager in production so two writable primaries cannot coexist.
- Plan WAL retention with replication slots and/or WAL archiving. This lab intentionally used neither a permanent slot nor WAL archiving.
- Protect credentials. The passwords shown here are lab-only values and must not be reused in production.
- Restrict security-group and pg_hba.conf access to required hosts only.
- Consider pg_rewind for a faster former-primary rejoin when its prerequisites are satisfied; this tested LAB-3 deliberately used a fresh pg_basebackup.
- Document application connection switching separately. Native PostgreSQL streaming replication does not by itself provide a virtual endpoint, automatic failover, or client redirection.
- Monitor pg_stat_replication on the primary and pg_stat_wal_receiver on the standby, plus disk/WAL growth and replication delay.

25. Official PostgreSQL References

- **PostgreSQL 18 - High Availability, Load Balancing, and Replication**
  https://www.postgresql.org/docs/18/high-availability.html
- **PostgreSQL 18 - Failover**
  https://www.postgresql.org/docs/18/warm-standby-failover.html
- **PostgreSQL 18 - System Administration / Recovery Control Functions**
  https://www.postgresql.org/docs/18/functions-admin.html
- **PostgreSQL 18 - Replication Configuration**
  https://www.postgresql.org/docs/18/runtime-config-replication.html
- **PostgreSQL 18 - Hot Standby**
  https://www.postgresql.org/docs/18/hot-standby.html
- **PostgreSQL 18 - Monitoring Database Activity**
  https://www.postgresql.org/docs/18/monitoring.html
- **PostgreSQL - pg_basebackup**
  https://www.postgresql.org/docs/current/app-pgbasebackup.html
- **PostgreSQL - pg_rewind**
  https://www.postgresql.org/docs/current/app-pgrewind.html